



Projet : Développement web full-stack

MeloStream : plateforme de streaming musical

Cas d'étude : recherche, écoute et gestion de playlists

Réalisé par :

Ayoub El Houani
Oussama Felouach
Mbarek Ettaleby

Filière : SDIA

Encadré par :

Mme Oumayma AGHERAI

Année universitaire : 2025/2026

Table des matières

Liste des figures	3
Liste des tableaux	4
Introduction	5
Contexte	5
Objectifs	5
Démarche de réalisation	6
1 Cahier des charges	7
1.1 Problématique	7
1.2 Besoins identifiés	7
1.2.1 Besoins fonctionnels	7
1.2.2 Besoins non fonctionnels	8
1.3 Solutions proposées	8
2 Conception fonctionnelle	9
2.1 Acteurs	9
2.2 Diagrammes de cas d'utilisation	9
2.2.1 Cas du visiteur	9
2.2.2 Cas de l'utilisateur	10
2.2.3 Cas de l'administrateur	10
2.3 Diagramme de classes	11
2.4 Organisation des données	12
2.5 Scénario de partage d'un titre	12
3 Conception technique	13
3.1 Architecture globale	13
3.2 Technologies utilisées	14
3.3 Structure du projet	14

3.4	Sécurité	15
3.5	Endpoints REST principaux	15
3.6	Documentation OpenAPI et Swagger	16
4	Réalisation	17
4.1	Fonctionnalités développées	17
4.1.1	Authentification	17
4.1.2	Recherche musicale	17
4.1.3	Lecture	17
4.1.4	Favoris	18
4.1.5	Playlists	18
4.1.6	Partage	18
4.1.7	Administration	19
4.1.8	Documentation API	19
4.1.9	Améliorations de l'interface	19
4.2	Captures d'écran	19
4.3	Validation de la réalisation	25
	Conclusion	27

Liste des figures

2.1	Diagramme de cas d'utilisation du visiteur	9
2.2	Diagramme de cas d'utilisation de l'utilisateur	10
2.3	Diagramme de cas d'utilisation de l'administrateur	10
2.4	Diagramme de classes principal	11
3.1	Architecture technique de MeloStream	13
4.1	Page d'authentification	20
4.2	Page d'inscription utilisateur	20
4.3	Accueil utilisateur et catalogue musical	21
4.4	Résultats d'une recherche de chanson	21
4.5	Message affiché après le partage d'un titre	22
4.6	Bibliothèque des titres favoris	22
4.7	Gestion des playlists utilisateur	23
4.8	Page des paramètres du profil	23
4.9	Interface d'administration	24
4.10	Documentation Swagger OpenAPI de l'API REST	25

Liste des tableaux

1.1	Besoins fonctionnels du projet	7
1.2	Besoins non fonctionnels du projet	8
2.1	Acteurs du système	9
3.1	Technologies utilisées dans le projet	14
3.2	Endpoints REST principaux	15
4.1	Validation de la réalisation	25

Introduction

Contexte

Les plateformes musicales occupent aujourd’hui une place importante dans les usages numériques. Elles permettent de rechercher des chansons, découvrir des artistes, organiser ses préférences et partager facilement des titres avec d’autres utilisateurs.

Le projet MeloStream consiste à développer une application web de streaming musical dans un contexte pédagogique. L’application propose une interface moderne côté client, une API REST sécurisée côté serveur et une base de données permettant de conserver les utilisateurs, les sessions et les titres favoris.

Le système utilise des services musicaux externes pour récupérer les chansons, les artistes, les albums, les pochettes et les liens audio. Deezer est utilisé par défaut pour les extraits audio, tandis que Jamendo peut être activé afin de proposer des titres libres ou licenciés.

Objectifs

Les objectifs du projet sont :

- créer une application web complète avec une séparation claire entre frontend et backend ;
- permettre l’inscription, la connexion, la déconnexion et la gestion du profil utilisateur ;
- proposer un catalogue musical consultable par recherche ou par humeur ;
- intégrer un lecteur audio sans lancer automatiquement la musique après la connexion ;
- permettre la sauvegarde des titres favoris ;
- permettre la création de playlists et l’organisation des titres ;
- ajouter une fonctionnalité de partage de titre par lien ;
- fournir une interface d’administration protégée par rôle ;
- produire une documentation claire pour comprendre, lancer et maintenir le projet.

Démarche de réalisation

La réalisation du projet a été organisée de manière progressive. Une première étape a consisté à définir les besoins principaux et les entités nécessaires : utilisateurs, jetons de connexion, favoris, playlists et titres musicaux. Le backend a ensuite été mis en place pour exposer une API REST sécurisée, avant d'intégrer l'interface Angular et les appels vers les services musicaux externes.

Le développement a suivi une logique fonctionnelle : chaque fonctionnalité visible dans l'interface correspond à un ensemble clair de routes backend, de services métier, de DTO et de méthodes côté frontend. Cette approche facilite la maintenance, car une évolution sur les favoris, les playlists ou l'administration reste localisée dans le domaine concerné.

Chapitre 1

Cahier des charges

1.1 Problématique

La problématique principale est de concevoir une plateforme musicale simple, ergonomique et sécurisée, capable de relier une interface Angular à une API Spring Boot et à une base de données MySQL. Le projet doit aussi communiquer avec des API musicales externes sans rendre le code difficile à maintenir.

L'application doit donc répondre à trois exigences majeures :

- fournir une expérience utilisateur fluide ;
- protéger les fonctionnalités privées par authentification ;
- garder une architecture technique claire et évolutive.

1.2 Besoins identifiés

1.2.1 Besoins fonctionnels

TABEAU 1.1 – Besoins fonctionnels du projet

Référence	Description
BF01	Créer un compte utilisateur.
BF02	Se connecter et se déconnecter.
BF03	Consulter et modifier son profil.
BF04	Rechercher des titres par mot-clé, artiste, album ou humeur.
BF05	Écouter un titre depuis un lecteur intégré.
BF06	Ajouter un titre aux favoris.
BF07	Supprimer un titre des favoris.
BF08	Créer, renommer et supprimer une playlist.
BF09	Ajouter ou retirer des titres d'une playlist.
BF10	Partager un titre avec un lien direct.

BF11	Charger automatiquement le titre ciblé lorsqu'un lien partagé est ouvert.
BF12	Accéder à un espace administrateur pour gérer les utilisateurs.
BF13	Enchaîner automatiquement les titres d'une playlist et naviguer avec précédent/suivant.
BF14	Présenter les actions musicales sous forme d'icônes claires avec des états visuels.

1.2.2 Besoins non fonctionnels

TABLEAU 1.2 – Besoins non fonctionnels du projet

Référence	Description
BNF01	L'interface doit être responsive et lisible.
BNF02	Les mots de passe doivent être stockés sous forme de hash.
BNF03	Les endpoints privés doivent être protégés par jeton.
BNF04	Les rôles doivent limiter l'accès à l'administration.
BNF05	Le code doit être organisé par domaine fonctionnel.
BNF06	La base de données doit être initialisée automatiquement en local.
BNF07	Le projet doit être testable et documenté.

1.3 Solutions proposées

La solution proposée repose sur :

- une application Angular pour l'interface utilisateur ;
- une API Spring Boot pour la logique métier et la sécurité ;
- une base MySQL pour la persistance ;
- une intégration Deezer/Jamendo pour les données musicales ;
- une authentification stateless par jeton ;
- une interface administrateur accessible seulement au rôle **ADMIN**.

Cette solution permet de séparer les responsabilités. Le frontend gère l'affichage, les formulaires, les interactions utilisateur et le lecteur audio. Le backend contrôle les règles métier, la persistance et la sécurité. La base MySQL conserve les données propres à l'application, tandis que les informations musicales sont récupérées à la demande depuis des services externes.

Chapitre 2

Conception fonctionnelle

2.1 Acteurs

TABLEAU 2.1 – Acteurs du système

Acteur	Rôle dans le système
Visiteur	Peut créer un compte ou se connecter.
Utilisateur	Peut rechercher, écouter, sauvegarder, organiser et partager des titres.
Administrateur	Peut gérer les utilisateurs et consulter les statistiques.
API musicale externe	Fournit les titres, artistes, albums, images et audios.

2.2 Diagrammes de cas d'utilisation

Les cas d'utilisation sont séparés par acteur afin de rendre la lecture plus claire. Chaque figure isole les actions accessibles à un profil donné, tandis que la généralisation de l'administrateur précise l'héritage des droits d'un utilisateur standard.

2.2.1 Cas du visiteur

MeloStream - Cas du visiteur

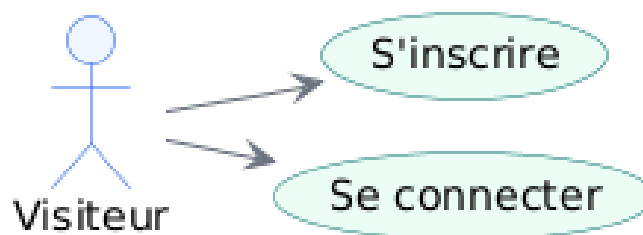


FIGURE 2.1 – Diagramme de cas d'utilisation du visiteur

- Le visiteur représente une personne qui n'est pas encore authentifiée.
- Il peut créer un compte afin d'accéder aux fonctionnalités privées de la plateforme.
- Il peut aussi se connecter lorsqu'il possède déjà un compte.

2.2.2 Cas de l'utilisateur

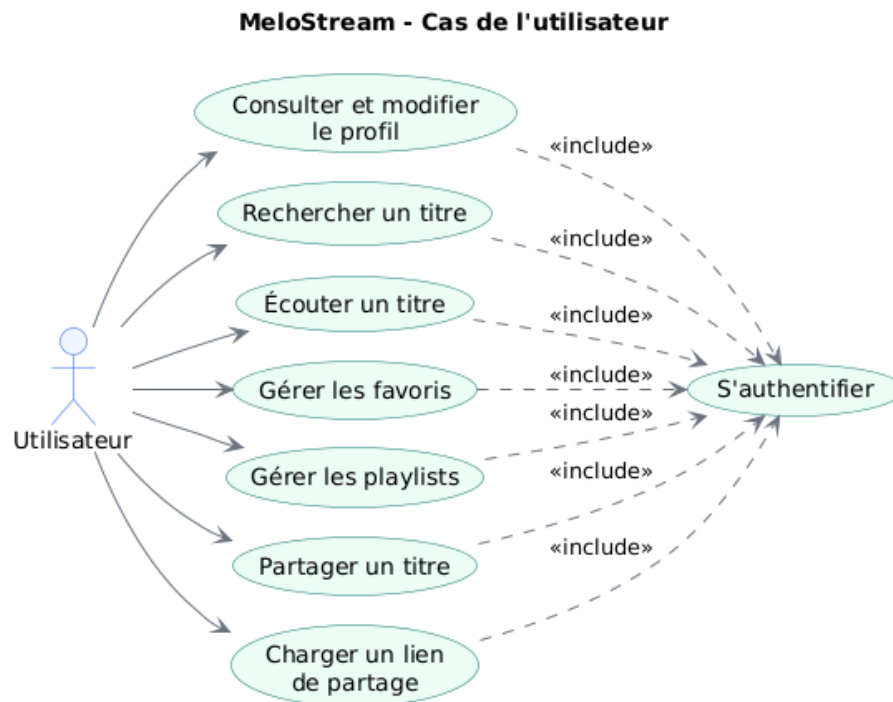


FIGURE 2.2 – Diagramme de cas d'utilisation de l'utilisateur

- L'utilisateur authentifié accède aux fonctionnalités musicales principales de l'application.
- Il peut rechercher et écouter des titres, gérer ses favoris et organiser ses playlists.
- Il peut partager un titre et ouvrir un lien de partage reçu depuis l'application.
- Il peut également consulter et modifier les informations de son profil.
- Les cas d'utilisation privés incluent **S'authentifier**, car ils nécessitent une session utilisateur valide.

2.2.3 Cas de l'administrateur

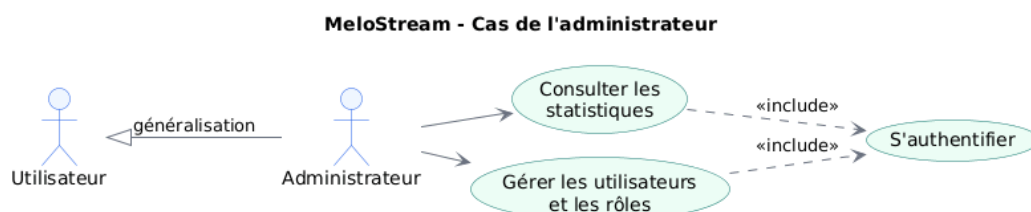


FIGURE 2.3 – Diagramme de cas d'utilisation de l'administrateur

- L'administrateur est une spécialisation de l'utilisateur.
- La flèche de généralisation indique qu'il conserve les fonctionnalités d'un utilisateur connecté.
- Il dispose en plus de fonctionnalités propres : consulter les statistiques et gérer les utilisateurs ainsi que leurs rôles.
- Les fonctionnalités d'administration incluent aussi **S'authentifier**, puis le contrôle du rôle **ADMIN** est appliqué par le backend.

2.3 Diagramme de classes

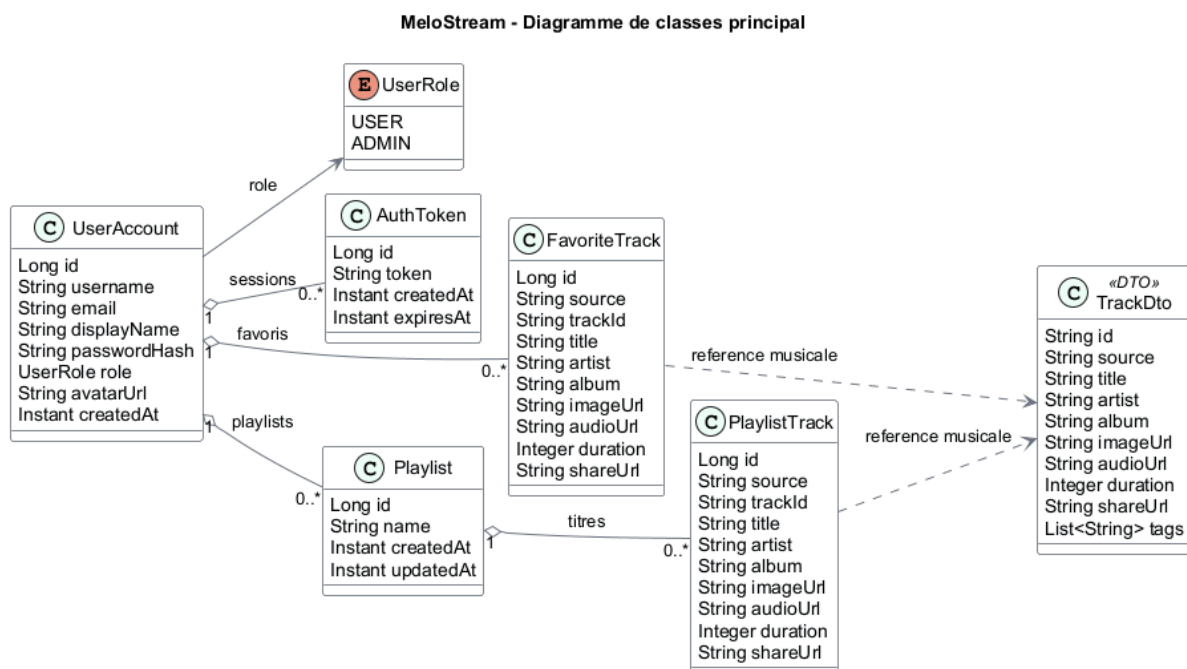


FIGURE 2.4 – Diagramme de classes principal

- Le diagramme décrit les entités principales utilisées par le backend.
- **UserAccount** représente le compte utilisateur avec son identifiant, son profil, son mot de passe hashé et son rôle.
- **AuthToken** conserve les jetons de session liés à un utilisateur pour authentifier les requêtes privées.
- Le champ **role** relie **UserAccount** à l'énumération **UserRole**. La valeur **USER** donne accès aux fonctions musicales, tandis que la valeur **ADMIN** active les fonctions d'administration.
- **FavoriteTrack** sauvegarde les titres favoris d'un utilisateur, tandis que **TrackDto** représente un titre normalisé venant d'une API musicale externe.
- **Playlist** et **PlaylistTrack** permettent d'organiser les chansons en listes personnalisées appartenant à chaque utilisateur.

2.4 Organisation des données

Les données de l'application sont réparties entre les données internes et les données externes. Les données internes concernent les comptes utilisateurs, les jetons d'authentification, les favoris et les playlists. Elles sont stockées dans MySQL afin de rester disponibles après le redémarrage de l'application.

Les données musicales, comme le titre, l'artiste, l'album, la pochette ou l'extrait audio, proviennent principalement des API Deezer et Jamendo. Le backend les transforme en objets `TrackDto` pour fournir au frontend un format unique, quelle que soit la source musicale utilisée. Cette normalisation évite de faire dépendre l'interface Angular du format exact de chaque API externe.

2.5 Scénario de partage d'un titre

Le partage est une fonctionnalité importante de l'application. Elle permet à un utilisateur de copier un lien vers un titre et à un autre utilisateur d'accéder directement au même titre.

1. L'utilisateur clique sur le bouton **Partager**.
2. Le frontend construit une URL au format `?song=source:id`.
3. Exemple : <http://localhost:4200/?song=deezer:3135556>.
4. Un autre utilisateur ouvre ce lien.
5. Après connexion, le frontend appelle l'API `/api/tracks/{source}/{trackId}`.
6. Le titre est chargé dans la zone de lecture rapide.
7. La musique ne démarre pas automatiquement.

Chapitre 3

Conception technique

3.1 Architecture globale

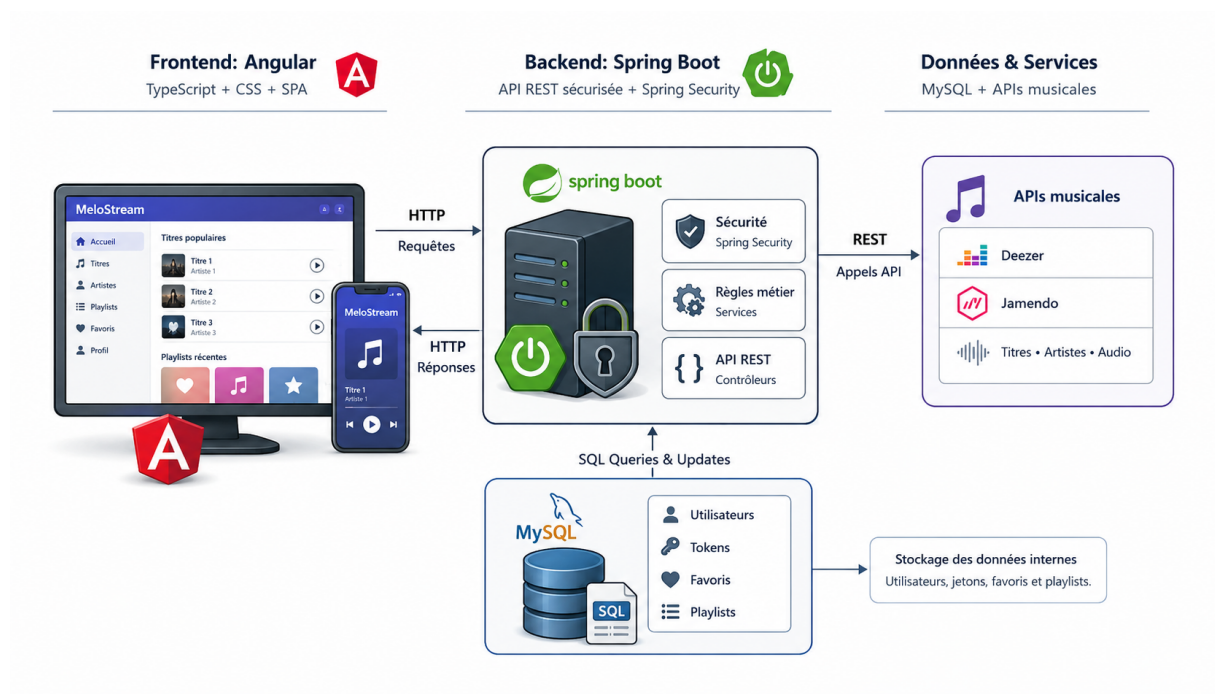


FIGURE 3.1 – Architecture technique de MeloStream

- Le diagramme montre le chemin suivi par les données entre l'utilisateur et les couches techniques du système.
- Le navigateur communique avec le frontend Angular, responsable de l'interface, des formulaires et des interactions utilisateur.
- En développement local, Angular redirige les requêtes `/api` vers le backend Spring Boot grâce au proxy.
- Le backend applique les règles métier, contrôle la sécurité et sert de point central entre le frontend, MySQL et les API musicales.
- MySQL stocke les données internes : utilisateurs, jetons, favoris et playlists.
- Les API Deezer et Jamendo fournissent les données musicales externes, comme les titres, artistes, pochettes et extraits audio.

- Les échanges entre les couches utilisent le format JSON, avec des DTO pour fournir au frontend des données normalisées.

3.2 Technologies utilisées

TABLEAU 3.1 – Technologies utilisées dans le projet

Couche	Technologies
Frontend	 Angular 21  TypeScript  CSS
Backend	 Java 21  Spring Boot  Spring MVC  Security  Data JPA
Base de données	 MySQL  phpMyAdmin
Documentation API	 OpenAPI  Swagger UI
APIs externes	 Deezer  Jamendo
Tests	 Spring Test  Angular

3.3 Structure du projet

```

project web/
  backend/
    src/main/java/com/musicstream/
      admin/
      auth/
      config/
      deezer/
      favorites/
      jamendo/
      music/
      security/
    src/main/resources/application.yml
  pom.xml
  frontend/
    src/app/
      admin-api.ts
      app.css
      app.html
      app.spec.ts
      app.ts

```

```
auth-api.ts
favorite-api.ts
music-api.ts
package.json
proxy.conf.json
rapport/
  rapport.tex
  rapport.pdf
README.md
```

3.4 Sécurité

La sécurité est assurée par Spring Security. L'application utilise un modèle stateless : après connexion, le backend génère un jeton stocké en base dans la table `auth_tokens`. Le frontend transmet ensuite ce jeton pour accéder aux endpoints protégés.

- `/api/auth/login` et `/api/auth/register` sont publics.
- `/swagger-ui.html`, `/swagger-ui/**` et `/v3/api-docs/**` sont publics pour consulter la documentation API.
- Les endpoints `/api/**` nécessitent une authentification.
- Les endpoints `/api/admin/**` nécessitent le rôle `ADMIN`.
- Les mots de passe sont hashés avant stockage.
- Les règles CORS autorisent le frontend local sur le port 4200.

Lorsqu'un utilisateur se connecte, le backend vérifie l'email et le mot de passe, puis crée un jeton temporaire valable plusieurs jours. Ce jeton est envoyé au frontend et stocké localement. Pour les requêtes privées, Angular l'ajoute dans l'en-tête `Authorization`. Le filtre de sécurité du backend lit ce jeton, retrouve l'utilisateur associé et autorise ou refuse l'accès selon son rôle.

Cette stratégie reste simple à utiliser dans un projet pédagogique tout en respectant les principes essentiels : les mots de passe ne circulent qu'au moment de la connexion, les routes sensibles sont protégées et les actions administrateur sont séparées des actions utilisateur.

3.5 Endpoints REST principaux

TABLEAU 3.2 – Endpoints REST principaux

Méthode	Endpoint	Description
POST	<code>/api/auth/register</code>	Créer un compte.
POST	<code>/api/auth/login</code>	Se connecter.
GET	<code>/api/auth/me</code>	Récupérer l'utilisateur connecté.

PUT	/api/auth/profile	Modifier le profil.
POST	/api/auth/logout	Se déconnecter.
GET	/api/tracks	Rechercher des titres.
GET	/api/tracks/{source}/ {trackId}	Charger un titre partagé.
GET	/api/favorites	Lister les favoris.
POST	/api/favorites	Ajouter un favori.
DELETE	/api/favorites/{trackId}	Supprimer un favori.
GET	/api/playlists	Lister les playlists.
POST	/api/playlists	Créer une playlist.
PUT	/api/playlists/ {playlistId}	Renommer une playlist.
DELETE	/api/playlists/ {playlistId}	Supprimer une playlist.
POST	/api/playlists/ {playlistId}/tracks	Ajouter un titre à une playlist.
DELETE	/api/playlists/ {playlistId}/tracks/ {source}/{trackId}	Retirer un titre d'une playlist.
GET	/api/admin/stats	Consulter les statistiques.
GET	/api/admin/users	Lister les utilisateurs.
PUT	/api/admin/users/ {userId}/role	Modifier un rôle.
DELETE	/api/admin/users/{userId}	Supprimer un utilisateur.

3.6 Documentation OpenAPI et Swagger

Le backend intègre Springdoc OpenAPI afin de générer automatiquement une documentation interactive à partir des controllers REST. La page Swagger UI est accessible localement après le démarrage du backend :

```
http://localhost:8080/swagger-ui.html
```

La spécification OpenAPI JSON est disponible à l'adresse suivante :

```
http://localhost:8080/v3/api-docs
```

Cette interface permet de consulter les endpoints, les schémas DTO et d'utiliser le bouton **Authorize** avec le jeton d'authentification retourné par l'endpoint de connexion.

Chapitre 4

Réalisation

4.1 Fonctionnalités développées

4.1.1 Authentification

L'utilisateur peut créer un compte, se connecter et se déconnecter. Après une connexion réussie, un jeton est utilisé pour sécuriser les requêtes suivantes.

Le formulaire d'authentification vérifie les informations saisies avant l'envoi au backend. Après une connexion réussie, l'utilisateur accède au tableau de bord musical avec son profil chargé depuis l'endpoint `/api/auth/me`. La déconnexion supprime le jeton local et invalide les jetons actifs côté serveur.

4.1.2 Recherche musicale

Le catalogue permet de rechercher des titres par mot-clé. L'utilisateur peut chercher un titre, un artiste ou un album. Le backend interroge les services musicaux externes et retourne des objets normalisés au frontend.

Les résultats affichent les informations utiles à l'écoute et à l'organisation des titres : nom de la chanson, artiste, album, image, durée et source. Cette présentation donne à l'utilisateur assez de contexte pour lancer la lecture, ajouter le titre aux favoris, le placer dans une playlist ou le partager.

Le champ de recherche conserve une interface compacte : le bouton de validation est représenté par une icône de loupe, accompagnée d'un libellé accessible pour les lecteurs d'écran et les infobulles.

4.1.3 Lecture

Chaque titre peut être lancé depuis l'interface. Une attention particulière a été portée au comportement après connexion : l'application ne lance pas la musique automatiquement.

Le lecteur reste donc contrôlé par l'utilisateur. Lorsqu'un titre est choisi, l'application met à jour la zone de lecture rapide et utilise l'URL audio fournie par l'API musicale. Si aucun extrait n'est disponible, l'interface peut continuer à afficher le titre sans déclencher de lecture.

Pour les playlists, le frontend maintient une file de lecture. Lorsqu'un titre se termine, le lecteur passe au titre suivant de la même playlist et relance la lecture dès que le nouvel extrait audio est prêt. Des boutons précédent et suivant permettent aussi de naviguer manuellement dans cette file. Si un lien audio sauvegardé devient indisponible, l'application tente de récupérer une version actualisée du titre via l'API avant de passer au morceau suivant.

Dans les cartes de titres, le bouton de lecture est intégré à l'image de couverture : une icône apparaît au survol ou au focus clavier. Cela garde les actions secondaires plus lisibles tout en conservant un accès direct à la lecture.

4.1.4 Favoris

Un utilisateur connecté peut sauvegarder un titre dans ses favoris et le retrouver plus tard. Les favoris sont persistés dans la base MySQL.

Chaque favori est rattaché au compte connecté. Cela permet d'avoir une bibliothèque personnelle différente pour chaque utilisateur. La contrainte d'unicité évite d'ajouter plusieurs fois le même titre dans les favoris d'un même compte.

L'état d'un favori est visible dans l'interface : le bouton coeur passe à une couleur accentuée et l'icône devient remplie lorsque le titre est déjà sauvegardé. L'utilisateur distingue ainsi immédiatement un titre enregistré d'un titre non enregistré.

4.1.5 Playlists

L'utilisateur peut créer des playlists personnalisées, les renommer, les supprimer et y ajouter des chansons depuis le catalogue ou la bibliothèque. Les titres d'une playlist peuvent ensuite être écoutés, partagés ou retirés.

Les playlists complètent les favoris en ajoutant une organisation par listes. Elles sont utiles pour regrouper des chansons par thème, humeur ou usage. Le backend vérifie que l'utilisateur connecté est bien propriétaire de la playlist avant d'autoriser les modifications.

La lecture depuis une playlist démarre soit depuis le premier titre de la liste, soit depuis le titre choisi par l'utilisateur. Dans les deux cas, le lecteur conserve le contexte de la playlist afin d'enchaîner automatiquement les titres suivants.

4.1.6 Partage

Chaque chanson possède un bouton de partage. Le lien généré contient la source et l'identifiant du titre. Exemple :

```
http://localhost:4200/?song=deezer:3135556
```

Lorsqu'un autre utilisateur ouvre ce lien, le titre est chargé après authentification.

Le partage ne stocke pas une copie complète du titre dans l'URL. Le lien contient seulement la source musicale et l'identifiant externe. Le backend peut alors recharger les informations du morceau via l'endpoint dédié et renvoyer au frontend un `TrackDto` complet.

4.1.7 Administration

Les comptes administrateurs peuvent consulter les statistiques, lister les utilisateurs, modifier les rôles et supprimer un utilisateur autre que leur propre compte.

L'espace administrateur est masqué pour les utilisateurs simples et protégé côté backend. Les statistiques donnent une vue rapide sur le nombre d'utilisateurs, de favoris et de playlists. Les actions sensibles, comme la suppression d'un compte ou la modification d'un rôle, passent par les endpoints `/api/admin/**`.

4.1.8 Documentation API

Swagger UI fournit une documentation interactive des endpoints REST du backend. Elle facilite la vérification des routes, des paramètres et des modèles échangés entre le frontend Angular et l'API Spring Boot.

4.1.9 Améliorations de l'interface

Plusieurs ajustements ont été apportés à l'ergonomie de l'interface :

- les actions répétées des cartes de titres et des lignes de playlist utilisent principalement des icônes afin de réduire l'encombrement visuel ;
- les boutons disposent de libellés accessibles avec `aria-label` et de titres affichés au survol ;
- le bouton de recherche est devenu une icône de loupe compacte ;
- les boutons de suppression utilisent un style plus doux, avec un fond clair et une couleur d'alerte moins agressive ;
- la zone *Lecture rapide* conserve des libellés texte pour les actions principales afin de rester explicite sur l'écran d'accueil.

4.2 Captures d'écran

Les captures suivantes ont été réalisées sur l'application locale. Elles présentent les principaux écrans développés : authentification, inscription, catalogue, recherche, partage, favoris, playlists, paramètres, administration et documentation Swagger.

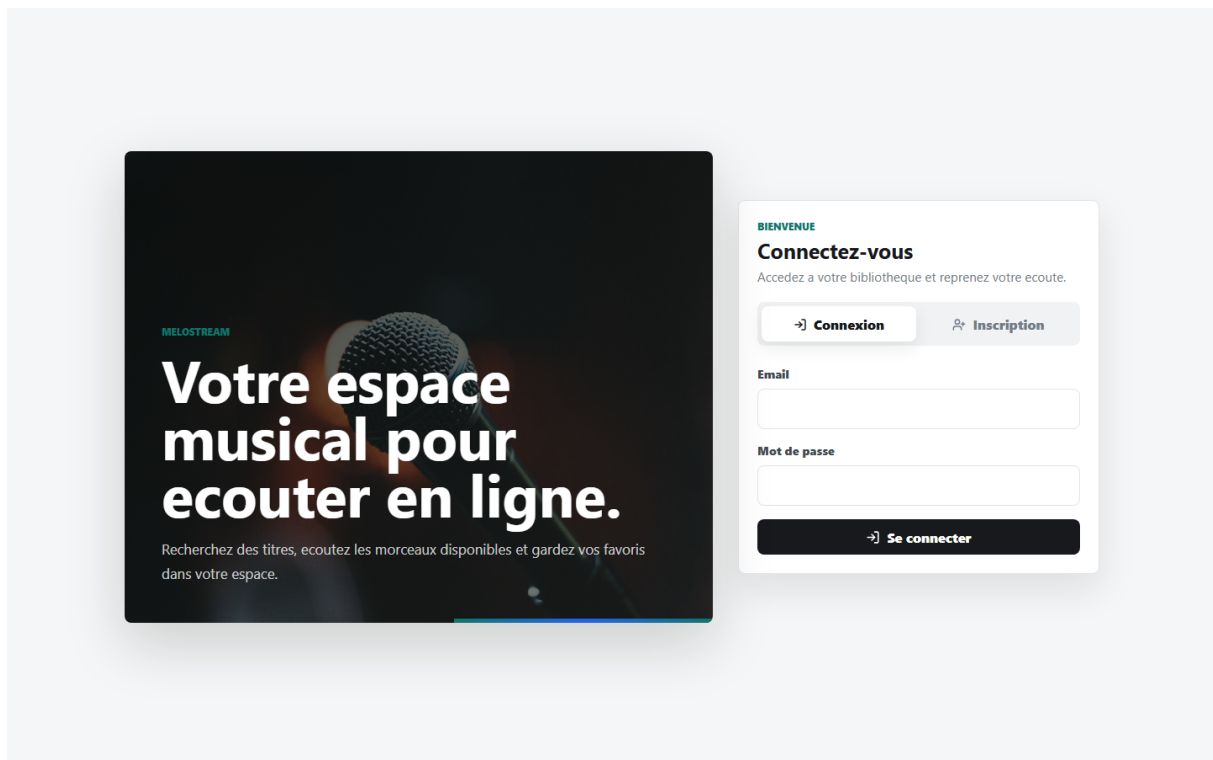


FIGURE 4.1 – Page d'authentification

Cette page permet à l'utilisateur de se connecter à son compte. Elle constitue le point d'entrée vers les fonctionnalités privées de l'application.

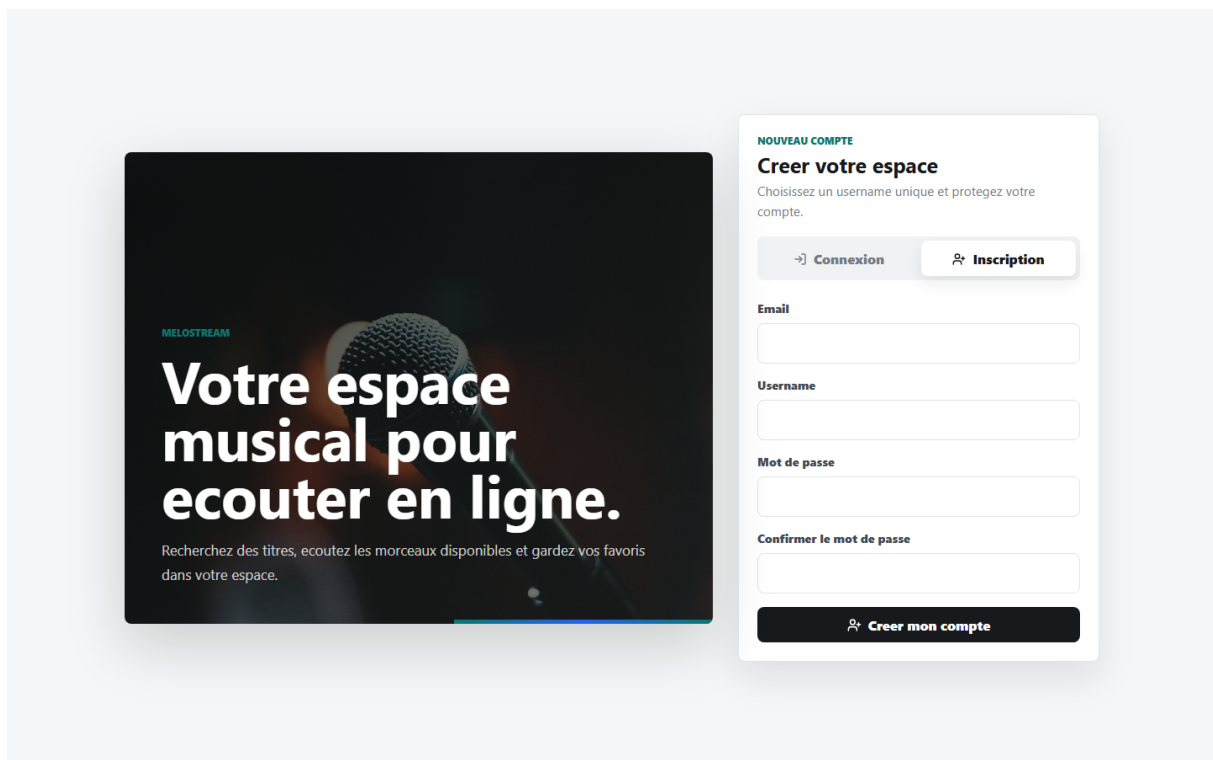


FIGURE 4.2 – Page d'inscription utilisateur

Cette interface permet de créer un nouveau compte utilisateur. Les informations saisies

sont envoyées au backend pour enregistrer le profil et générer une première session.

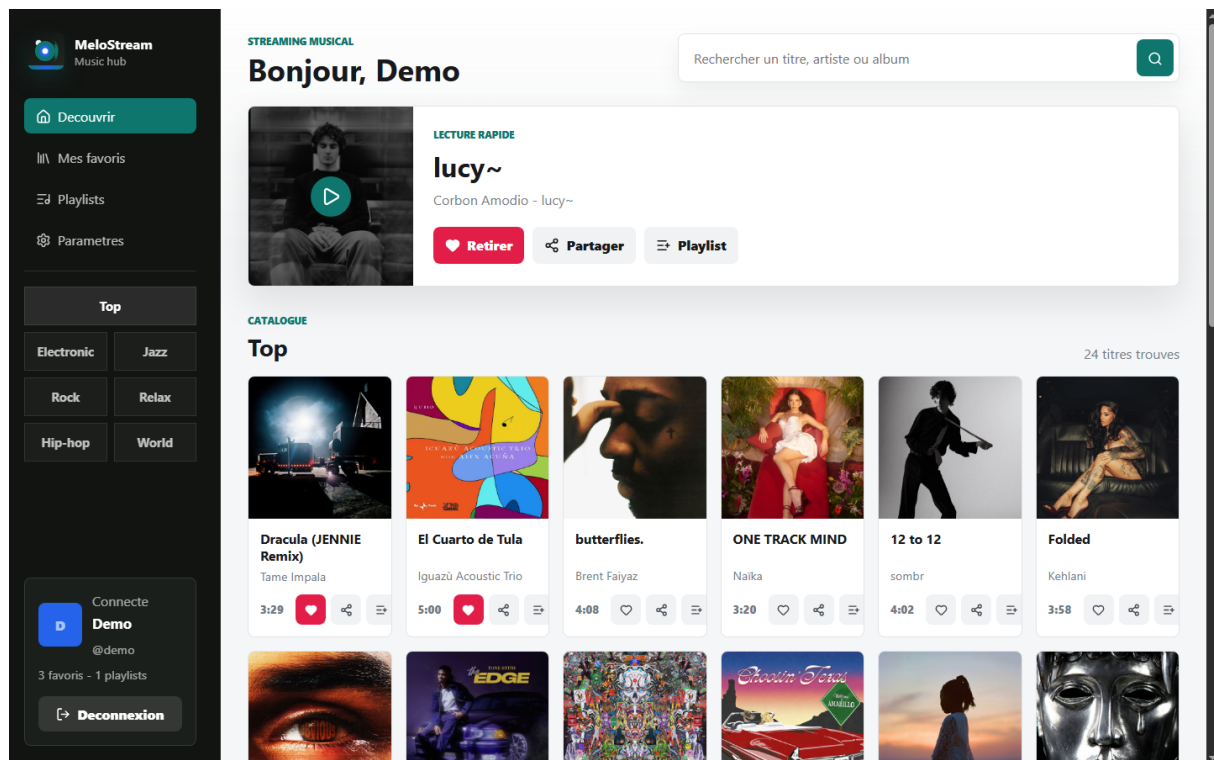


FIGURE 4.3 – Accueil utilisateur et catalogue musical

L'écran d'accueil présente le catalogue musical après connexion. L'utilisateur peut parcourir les titres proposés, lancer une écoute ou accéder aux actions rapides.

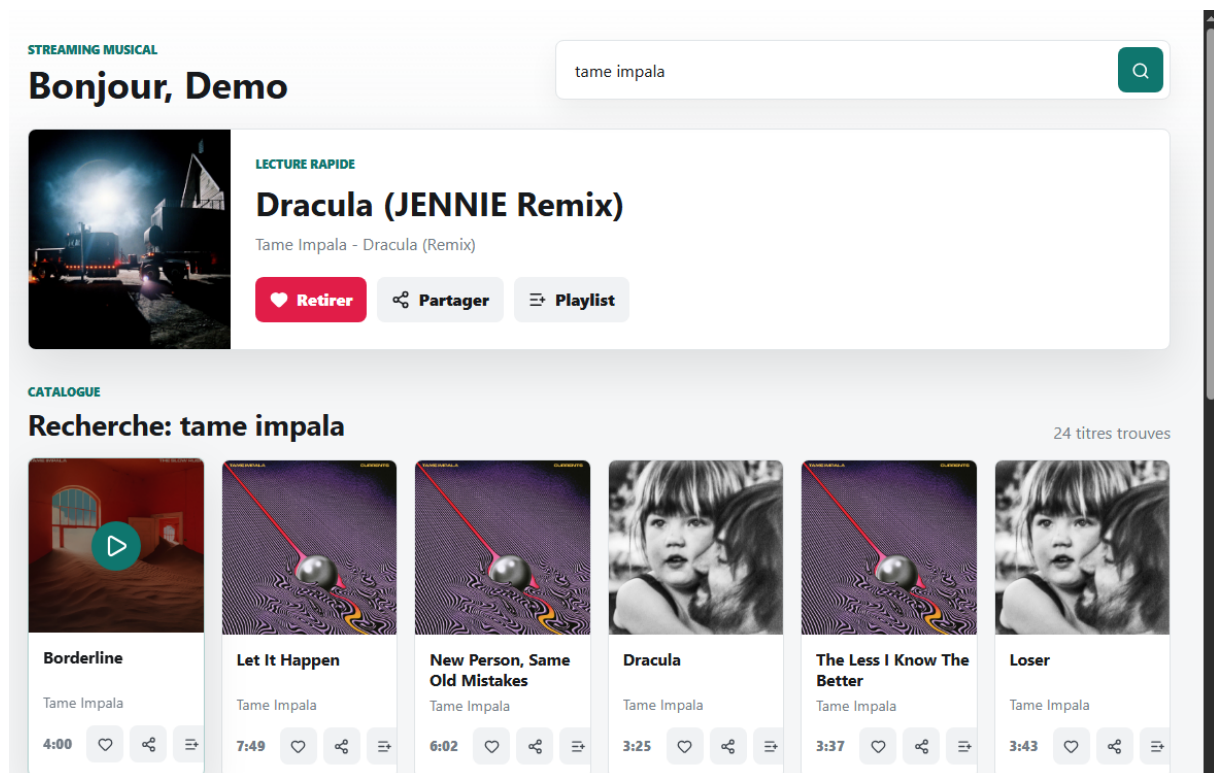


FIGURE 4.4 – Résultats d'une recherche de chanson

Cette capture montre le résultat d'une recherche musicale. Les titres retournés par l'API sont affichés avec les informations nécessaires : artiste, album, image et actions disponibles.

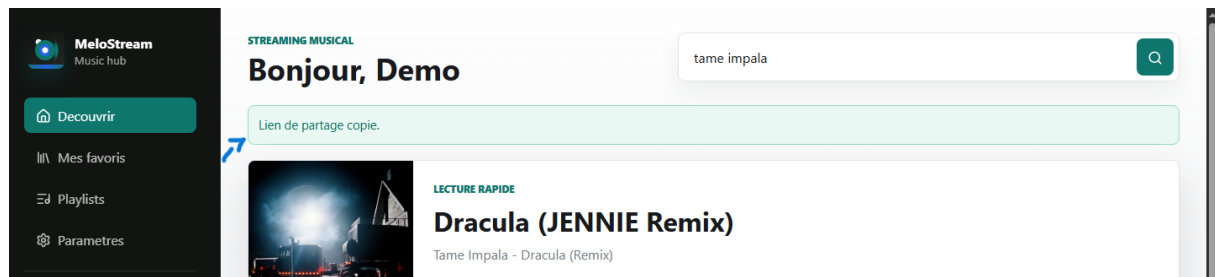


FIGURE 4.5 – Message affiché après le partage d'un titre

Le message confirme que le lien de partage du titre a été généré. Ce lien peut être transmis à un autre utilisateur pour ouvrir directement le même morceau.

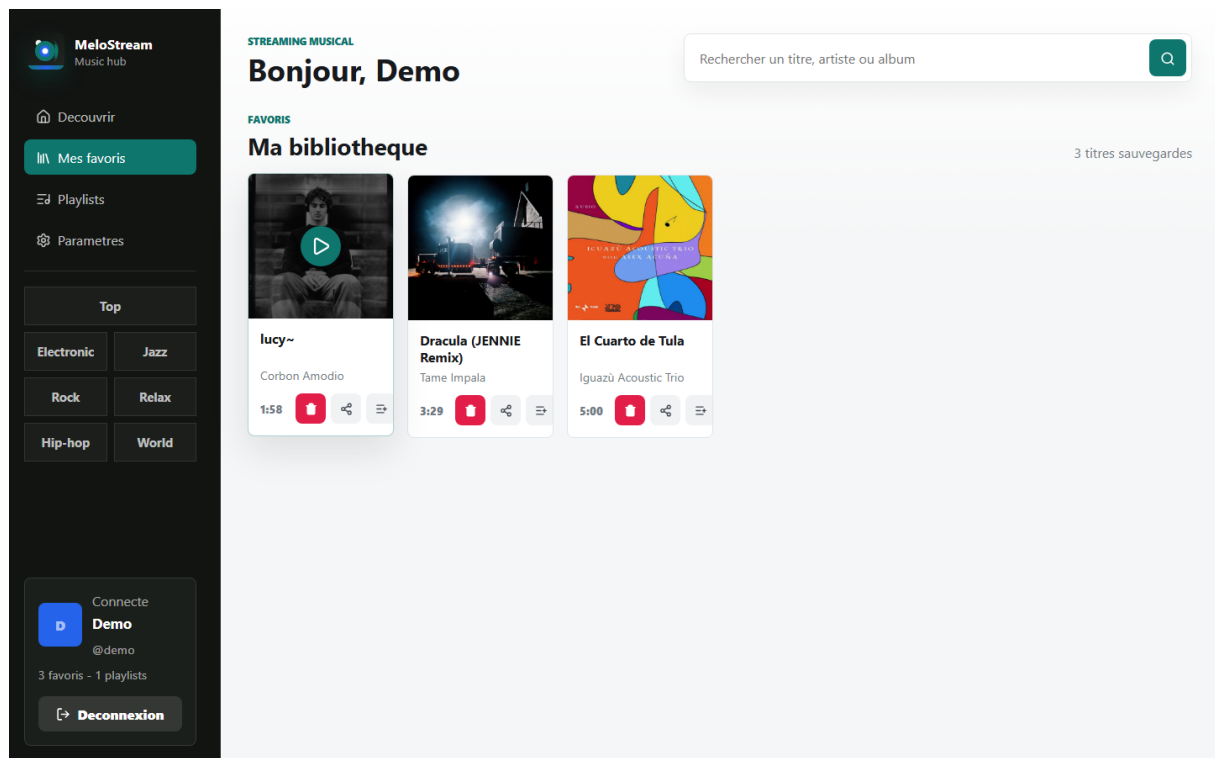


FIGURE 4.6 – Bibliothèque des titres favoris

Cette page regroupe les titres sauvegardés par l'utilisateur. Elle permet de retrouver rapidement ses morceaux préférés et de les gérer depuis un espace personnel.

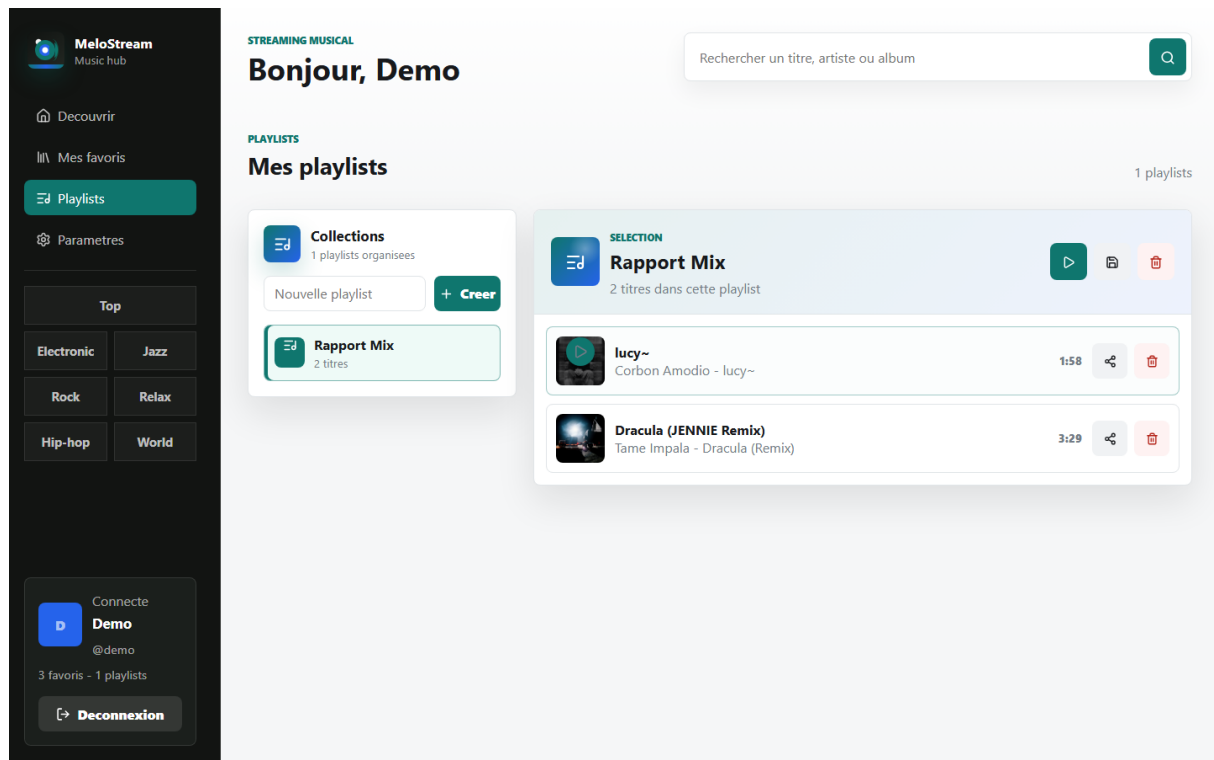


FIGURE 4.7 – Gestion des playlists utilisateur

L'écran des playlists permet d'organiser les chansons en listes personnalisées. L'utilisateur peut créer une playlist, y ajouter des titres ou retirer ceux qui ne sont plus souhaités.

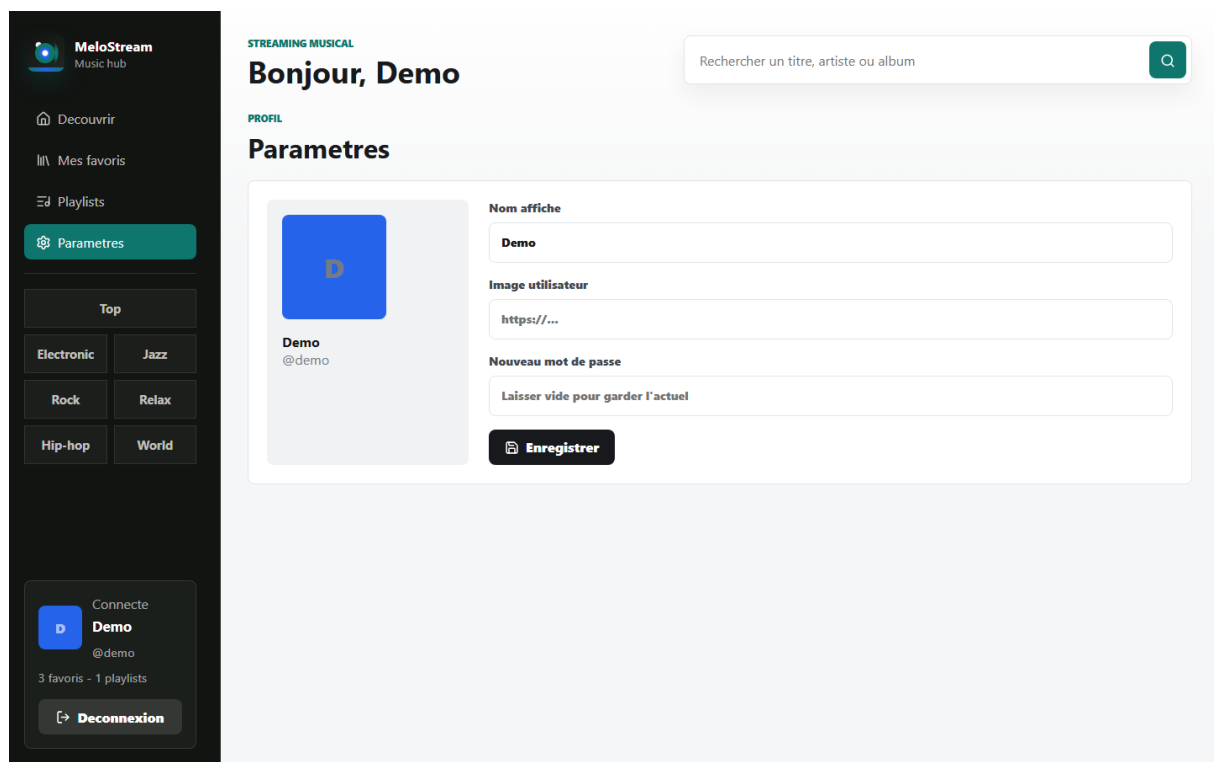


FIGURE 4.8 – Page des paramètres du profil

Cette page permet de modifier les informations du profil, comme le nom d'affichage, l'ava-

tar ou le mot de passe. Elle donne à l'utilisateur le contrôle sur ses données personnelles.

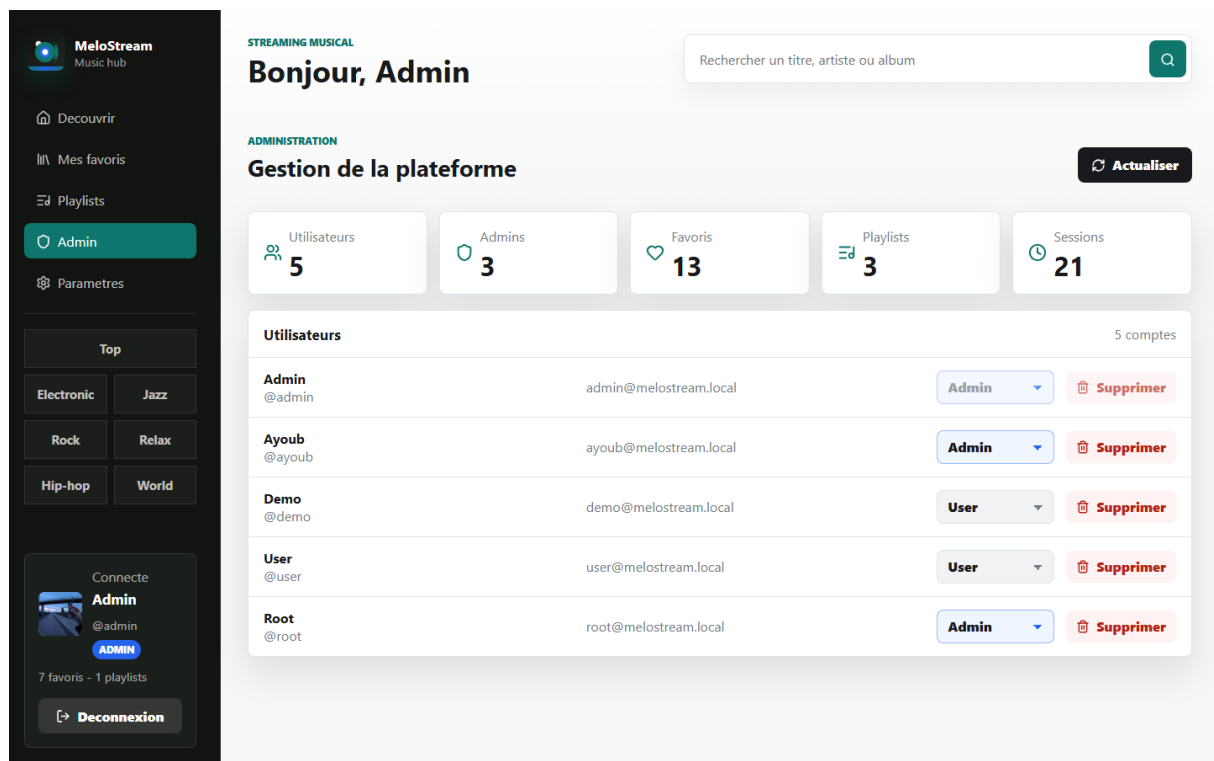


FIGURE 4.9 – Interface d'administration

Cette interface est réservée aux comptes administrateurs. Elle permet de consulter les statistiques et de gérer les utilisateurs de l'application.

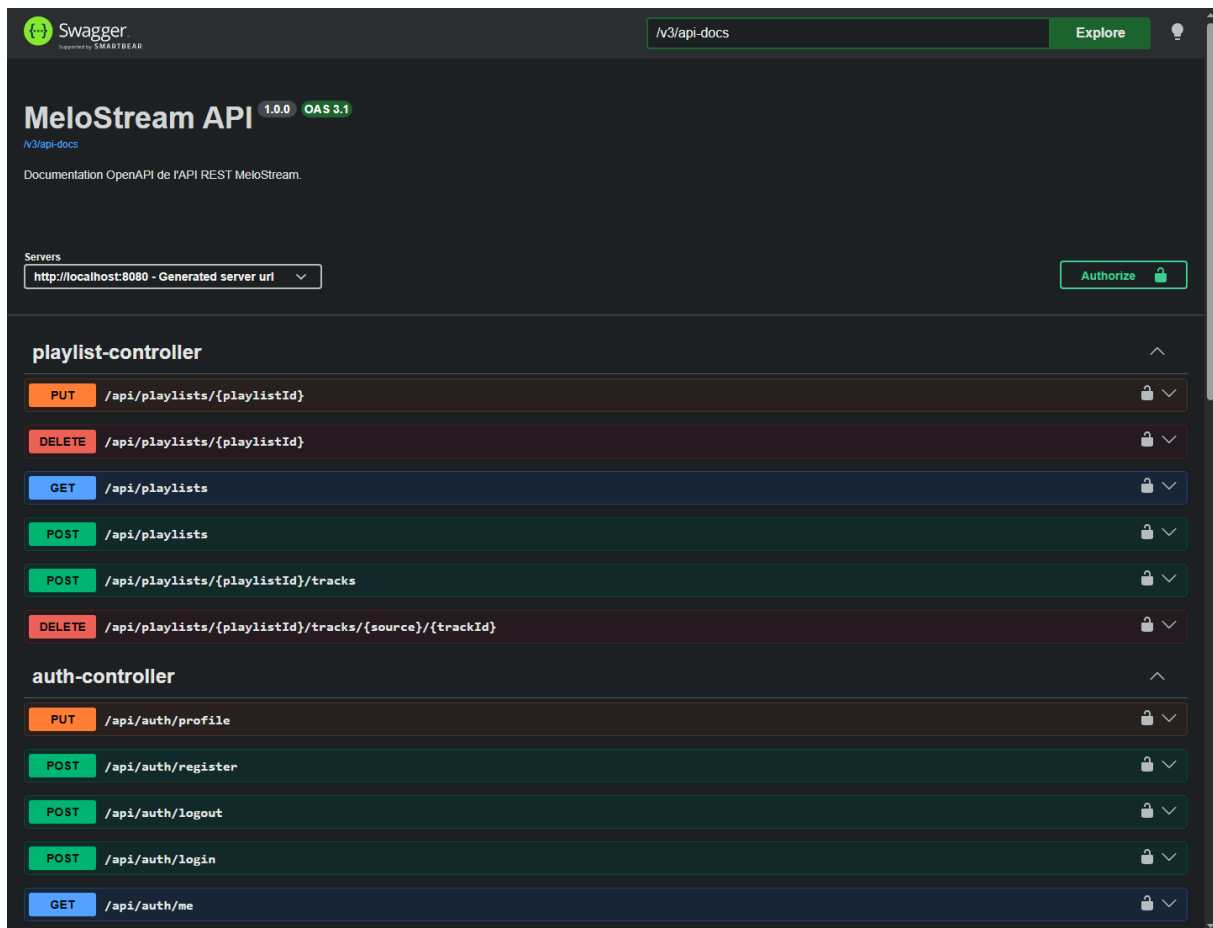


FIGURE 4.10 – Documentation Swagger OpenAPI de l'API REST

Swagger UI présente automatiquement la documentation des endpoints REST. Cette page facilite le test des routes et la compréhension des données échangées entre frontend et backend.

4.3 Validation de la réalisation

La validation a été effectuée sur les deux parties du projet afin de vérifier la stabilité de l'application et la cohérence entre le frontend et le backend.

TABLEAU 4.1 – Validation de la réalisation

Validation	Objectif
Tests backend	Vérifier le chargement du contexte Spring Boot, la configuration principale et l'utilisation du profil de test dédié.
Tests frontend	Vérifier le comportement des composants Angular et les scénarios principaux de l'interface.

Build frontend	Confirmer que l'application Angular se compile correctement, que les imports TypeScript sont valides et que les fichiers de production sont générés.
Documentation API	Vérifier que Swagger UI expose les endpoints REST et les DTO nécessaires à la compréhension de l'API.

Ces vérifications confirment que les fonctionnalités principales développées peuvent être utilisées ensemble : authentification, recherche musicale, lecture, favoris, playlists, partage, administration et documentation API.

Conclusion

Résumé du projet

Le projet MeloStream répond aux objectifs fixés dans le cahier des charges : il propose une plateforme musicale complète avec authentification, recherche, lecture, favoris, playlists, partage et administration. L'architecture sépare clairement le frontend Angular, le backend Spring Boot et la base MySQL, ce qui facilite la maintenance et les évolutions futures.

Défis rencontrés

Les principaux défis rencontrés concernent l'intégration des API musicales externes, la disponibilité variable des liens audio, la gestion des liens de partage et la sécurisation des endpoints privés. La synchronisation du lecteur audio avec les playlists a également nécessité une attention particulière afin d'assurer le passage automatique au titre suivant et la navigation précédent/suivant.

Recommandations et perspectives

Pour améliorer et maintenir la solution, il est recommandé d'ajouter davantage de tests d'intégration, de préparer une configuration de déploiement avec des variables d'environnement séparées, d'ajouter une route Angular dédiée au partage et d'améliorer l'observabilité de l'application. À plus long terme, la plateforme pourrait intégrer une pagination avancée du catalogue, une gestion plus complète des playlists et une stratégie de cache pour limiter les appels aux APIs musicales externes.